

Self-Attention Step by Step

Contents

1. Overview	2
2. Prerequisites	2
3. The idea: a word as a mix of its sentence	3
4. Weights from similarity	3
4.1 Dot products as similarity	3
4.2 Softmax turns scores into weights	4
4.3 The whole computation for one word	4
5. All words at once, with matrices	5
6. The problem: nothing to learn	6
7. Three roles: query, key and value	7
8. Query, key and value vectors from learned matrices	8
8.1 A linear transformation per role	8
8.2 The refined computation	9
8.3 All words at once	10
9. Learned matrices on real reviews	10
9.1 The experiment	11
9.2 Results	11
9.3 What the weights learned	11
10. Summary	14
11. Sources	14
12. Key terms	15

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Attention mechanism ([Note 1069](#)).
- **Leads to:** Scaled dot-product attention ([Note 1074](#)); Positional encoding ([Note 1078](#)); Masked self-attention ([Note 1081](#)).
- **Compare with:** Cross-attention ([Note 1082](#)).

1. Overview

Key point: Self-attention builds each word’s new vector as a **weighted sum** of the embeddings of all the words in the sentence. The weights come from **dot products** (how similar two words are), normalised by a **softmax**. Each embedding plays three roles, **query**, **key** and **value**, so three learned matrices W_Q , W_K , W_V turn it into three vectors, one per role. For a whole sentence at once: $Y = \text{softmax}(QK^\top) V$.

The [what is self-attention Note](#) treated self-attention as a box: static embeddings go in, contextual embeddings come out. This Note opens the box. We build self-attention from first principles, as if inventing it:

1. a simple version with no parameters (sections 3 to 5);
2. its two properties: it runs in parallel, and it cannot learn (section 6);
3. the fix: query, key and value vectors from learned matrices (sections 7 and 8);
4. a test on real movie reviews: fixed against learned (section 9).

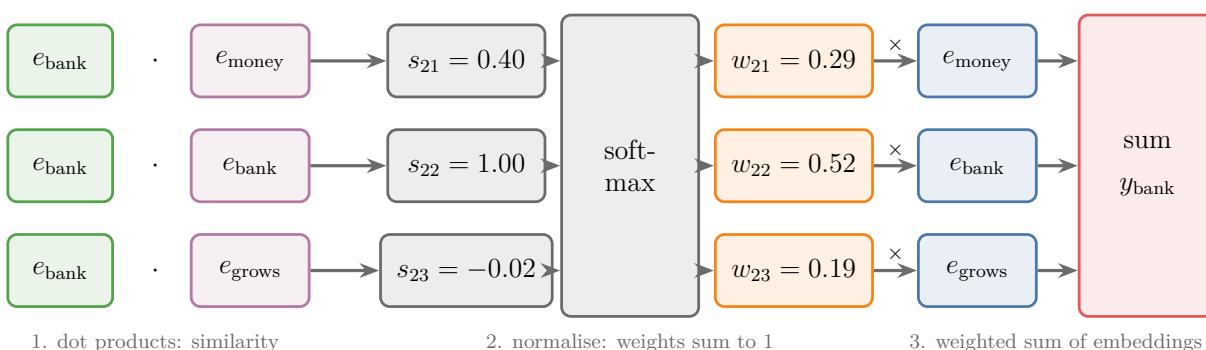


Figure 1: The first-principles computation of the new vector for “bank” in “money bank grows”, with the real numbers of the Notebook: dot products, softmax, weighted sum

2. Prerequisites

- [What is self-attention](#): static and contextual embeddings.
- [Dot product and cosine similarity](#): the dot product as a measure of similarity.

- **Softmax regression**: softmax turns any numbers into positive weights that sum to 1.
- **Matrix multiplication as composition** and **linear transformations**: a vector times a matrix is a new vector.

3. The idea: a word as a mix of its sentence

Key point: Write each word’s new meaning as a mix of all the words of its sentence. “Bank” in “money bank grows” becomes mostly bank, some money, a little grows.

Take two short phrases, “money bank grows” and “river bank flows”. “Bank” means something different in each, but a static embedding gives it the same vector in both. Suppose we describe each word not by itself but as a mix of the words around it:

$$\text{bank}_{\text{new}} = 0.29 \text{ money} + 0.52 \text{ bank} + 0.19 \text{ grows}$$

$$\text{bank}_{\text{new}} = 0.25 \text{ river} + 0.54 \text{ bank} + 0.21 \text{ flows}$$

The left-hand sides are the same word, but the right-hand sides differ, because the neighbours differ. The meaning of “bank” now depends on its context. We can do the same for every word of each phrase: “money” becomes a mix of money, bank and grows, and so on.

Machines work with vectors, not words, so we write the mix with embeddings. With e_{money} the embedding of “money” (a vector of n numbers), the new vector of “bank” is

$$y_{\text{bank}} = 0.29 e_{\text{money}} + 0.52 e_{\text{bank}} + 0.19 e_{\text{grows}}$$

A weighted sum of n -number vectors is again an n -number vector. The new vectors are the **contextual embeddings** we wanted. Two questions remain: where do the weights come from, and how do we make them sum to 1?

4. Weights from similarity

Key point: A weight says how much one word should borrow from another, so it should measure how similar the two words are. The dot product of their embeddings measures similarity; the softmax turns the dot products into weights that sum to 1.

4.1 Dot products as similarity

Key point: Two vectors that point the same way have a large dot product.

The weight 0.29 means: “bank” takes 29% of its new meaning from “money”. The more related two words are, the larger that share should be. The **dot product** of two vectors is a simple measure of how similar they are (the **dot product Note**).

1. **In words:** multiply the vectors number by number, then add.
2. **Formula:** $a \cdot b = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$.

3. **Example:** with $a = (6, 1)$, $b = (4, 2)$ and $c = (1, 5)$:

$$a \cdot b = 6 \times 4 + 1 \times 2 = 26, \quad a \cdot c = 6 \times 1 + 1 \times 5 = 11$$

a is more similar to b than to c , and the dot products say so.

In the Notebook every embedding has length 1, so the dot product is exactly the cosine similarity: 1 for a word with itself, near 0 for unrelated words.

4.2 Softmax turns scores into weights

Key point: Raw dot products can be any size and even negative. The softmax makes them positive and makes them sum to 1.

Call $s_{ij} = e_i \cdot e_j$ the **score** of word i with word j . Scores can be large, small or negative, for example 36, -15 and 32. We want weights that are positive and sum to 1, so they read as shares: “bank is 52% bank, 29% money, 19% grows”. The **softmax** function does exactly this.

1. **In words:** raise e to the power of each score, then divide by the total.
2. **Formula:** for word i in a sentence of N words,

$$w_{ij} = \frac{e^{s_{ij}}}{e^{s_{i1}} + e^{s_{i2}} + \dots + e^{s_{iN}}}$$

3. **Example:** the real scores of “bank” (word 2) in “money bank grows”, from the Notebook’s embeddings, are $s_{21} = 0.40$ (with money), $s_{22} = 1.00$ (with itself) and $s_{23} = -0.02$ (with grows):

$$e^{0.40} = 1.49, \quad e^{1.00} = 2.72, \quad e^{-0.02} = 0.98, \quad \text{total} = 5.19$$

$$w_{21} = \frac{1.49}{5.19} = 0.29, \quad w_{22} = \frac{2.72}{5.19} = 0.52, \quad w_{23} = \frac{0.98}{5.19} = 0.19$$

The weights sum to 1, and they are the numbers of section 3.

4.3 The whole computation for one word

Key point: Three steps: dot products with every word, softmax, weighted sum.

Figure 1 puts the steps together for “bank”:

1. **Scores:** the dot product of e_{bank} with each embedding of the sentence gives s_{21}, s_{22}, s_{23} .
2. **Weights:** the softmax of the scores gives w_{21}, w_{22}, w_{23} , which sum to 1.
3. **Output:** the weighted sum $y_{\text{bank}} = w_{21} e_{\text{money}} + w_{22} e_{\text{bank}} + w_{23} e_{\text{grows}}$.

The same three steps, with “money” and then “grows” in place of “bank”, give y_{money} and y_{grows} . This simple version of attention, scores by dot product, softmax, weighted sum, is the one SLP3 (§7.1) presents before adding parameters.

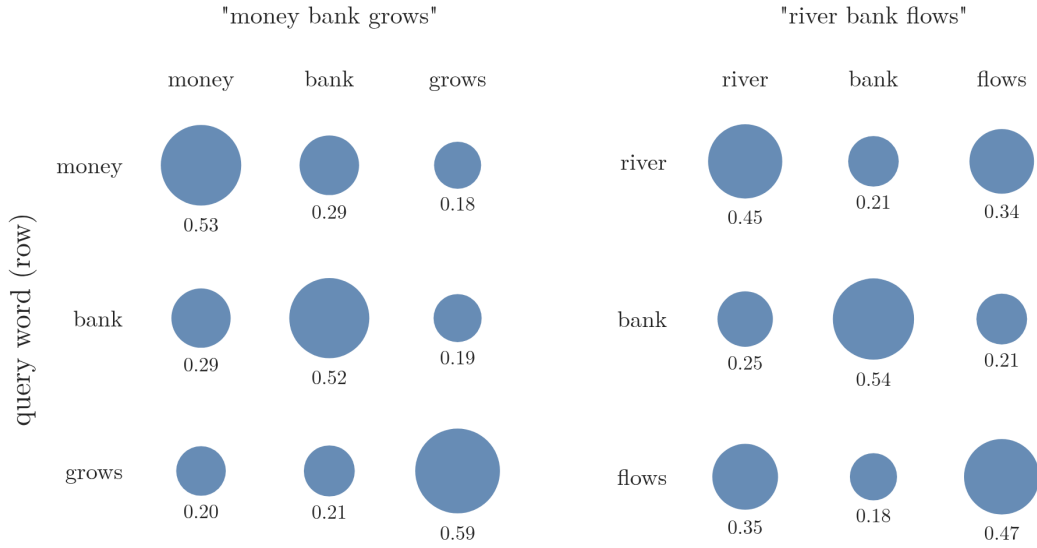


Figure 2: The weights of the simple self-attention for both phrases, with real embeddings, as a grid of dots: the area of each dot is the weight. Row i shows how word i mixes the words of its phrase; every row sums to 1

Figure 2 shows all the weights, one dot per pair of words, so the large ones stand out at a glance (a picture of the weights used by Sanderson 2024, Ch 6). Each word keeps about half of its own meaning (the diagonal: a vector is most similar to itself) and borrows the rest from its neighbours. The row of “bank” differs between the two phrases: it borrows 0.29 from “money” in one, 0.25 from “river” in the other. In the previous Note, these outputs moved “bank” towards “river” in “river bank flows”.

5. All words at once, with matrices

Key point: Stack the embeddings as the rows of a matrix X . Then $XX^\top$ holds every score, a row-wise softmax gives every weight, and multiplying by X gives every output: $Y = \text{softmax}(XX^\top) X$.

Nothing in the computation for “grows” waits for the computation for “bank”: each word’s new vector needs only the embeddings, which are all known at the start. So all the words can be processed at the same time, using matrices.

1. **In words:** put the embeddings of the N words as the rows of a matrix; multiply it by its own transpose to get all the scores; apply the softmax to each row; multiply the weights by the embeddings.
2. **Formula:** with X of shape $N \times n$ (one row per word),

$$S = XX^\top \ (N \times N), \quad W = \text{softmax}_{\text{rows}}(S), \quad Y = WX \ (N \times n)$$

Entry (i, j) of S is $e_i \cdot e_j = s_{ij}$, and row i of Y is y_i .

3. **Example:** for “money bank grows”, X is 3×50 (3 words, 50 numbers each), S and W are 3×3 (Figure 2, left) and Y is 3×50 . For “i put my money in the bank”, X is 7×50 , S is 7×7 and Y is again 7×50 . In the Notebook, the matrix version gives exactly the same numbers as a loop over the words, one at a time.

Whether the sentence has 3 words or 3,000, the work is a few matrix multiplications, which a GPU does in parallel. This parallelism is what lets transformers train fast (the [introduction to transformers Note](#), section 5).

The parallel computation has a cost. Nothing in $Y = \text{softmax}(XX^\top)X$ depends on the order of the rows: shuffle the words and each word gets the same new vector, only in a different row. The order of words is lost, although word order matters in text. The transformer restores it separately, with positional encoding (the [positional encoding Note](#)).

6. The problem: nothing to learn

Key point: The simple version has no parameters. Its contextual embeddings are general: the same for every task and every dataset. A task often needs **task-specific** contextual embeddings, so we must add learnable weights.

The three steps are a dot product, a softmax and another product. None of them has a weight or a bias. With no parameters there is nothing to train: the output for a sentence depends only on the static embeddings, never on the task or its data.

The resulting contextual embeddings are **general**: they use the context, but in the same way for every task. Sometimes that is not enough. Suppose we train an English-to-Hindi translator whose data contains

- “how are you” $\rightarrow$ “main theek hoon” (as a reply), and
- “that task is a piece of cake” $\rightarrow$ “vah kaam bahut aasaan hai” (“that work is very easy”).

General contextual embeddings of “piece of cake” lean towards the literal meaning, a slice of cake. A good translator must learn from its data that the phrase means “very easy”. In the same way, “break a leg” should become “shubh kaamnaayein” (“good wishes”) and not a sentence about a broken leg. What the model needs are **task-specific contextual embeddings**: still built from the neighbouring words, but in a way learned from the task’s data. For that, self-attention needs learnable parameters.

Where can they go? The softmax is a fixed formula. The parameters can only enter in the two products: the one that computes the scores and the one that forms the weighted sum.

7. Three roles: query, key and value

Key point: In the computation, every embedding is used three times, in three roles: as the **query** that asks “how similar are you to me?”, as the **key** that is compared with the query, and as the **value** that is mixed into the output.

Look again at Figure 1. The embedding of “bank” appears in three different places:

- **Query** (green): e_{bank} asks every word of the sentence, “how similar are you to me?”. It is the word whose new vector we are computing.
- **Key** (purple): $e_{\text{money}}, e_{\text{bank}}, e_{\text{grows}}$ answer: their dot product with the query gives the scores. In the computation for “money” or “grows”, e_{bank} also serves as a key.
- **Value** (blue): the same embeddings are mixed with the weights to form the output.

The names come from dictionaries in programming. A Python dictionary $d = \{\text{"a"}: 2, \text{"b"}: 3, \text{"c"}: 4\}$ stores **keys** (“a”, “b”, “c”) with their **values** (2, 3, 4). The lookup $d[\text{"a"}]$ is a **query**: it is compared with every key, and the value of the matching key, 2, is returned. Self-attention is a soft version of this lookup: the query is compared with every key, and instead of returning one value, it returns a mix of all the values, weighted by how well each key matches. SLP3 (§7.1) describes the same three roles.

The problem with the simple version is that one vector, the static embedding, plays all three roles. An analogy shows why that is a poor design. A writer is looking for a partner on a matrimonial website. His full life story is in his 300-page autobiography: his embedding. The website needs three things from him:

- a **profile** that others see when they search (his key);
- a **search** describing whom he is looking for (his query);
- what he says once there is a **match** (his value).

Uploading the whole autobiography as profile, search and first message would work badly in all three places. Instead he writes three short, focused texts, each drawn from the autobiography but shaped for its job. And he learns from experience what to put in each. If he gets matched with people who love politics when he does not, he removes “I write about politics” from his profile; if a broad search works better than a narrow one, he widens it. Each version is tuned from the results, that is, from data.

Self-attention should do the same: from each embedding, derive three separate vectors, a query vector, a key vector and a value vector, and learn from the training data how to derive them.

What a query and a key could compute: an invented example. Suppose the model would like nouns to take in the adjectives in front of them, as in “the old brown dog”. Imagine that each noun’s query asks “are there adjectives before me?” and that each adjective’s key answers “I am an adjective here”. If the matrices put such questions and answers into the same direction, a large dot product means a match (Sanderson 2024, Ch 6, who builds the same kind of imagined example). With 2 numbers per query and key:

1. **In words:** the query of “dog” points along the first axis (looking for adjectives); the

keys of “old” and “brown” point mostly the same way; the keys of “the” and “dog” point along the second axis.

2. **Formula:** $s_{extdog,j} = q_{extdog} \cdot k_j$, then the softmax over j .
3. **Example:** $q_{extdog} = (3, 0)$, $k_{extthe} = (0, 2)$, $k_{extold} = (3, 0.5)$, $k_{extbrown} = (2.8, 1)$, $k_{extdog} = (0.2, 2)$. The scores are 0, 9, 8.4 and 0.6, and the softmax gives the weights 0.000, 0.646, 0.354 and 0.000: “dog” takes its new meaning from “old” and “brown” only.

In a trained model nobody chooses these directions, and what a matrix actually learns is much harder to read; the example only shows the kind of job a query and a key can do together. Section 9.3 shows a real one: learned matrices that make every word look for the sentiment word.

8. Query, key and value vectors from learned matrices

Key point: Multiply each embedding by three matrices: $q = eW_Q$, $k = eW_K$, $v = eW_V$. The same three matrices serve every word. They start random and are trained by backpropagation, so the contextual embeddings become task-specific.

8.1 A linear transformation per role

Key point: A new vector from an old one: multiply by a matrix.

To make a new vector from an old one, scaling alone (making it longer or shorter) is too limited: the direction must change as well. The standard way is a **linear transformation**, multiplying the vector by a matrix (the [linear transformations Note](#)). We use three matrices, W_Q , W_K and W_V , one per role.

1. **In words:** each embedding, written as a row vector, times each of the three matrices gives the word’s query, key and value vectors.
2. **Formula:**

$$q_i = e_i W_Q, \quad k_i = e_i W_K, \quad v_i = e_i W_V$$

which is the form of SLP3 (eq. 7.9).

3. **Example:** in 2-D, with $e = (1, 2)$,

$$W_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad W_V = \begin{pmatrix} 2 & 0 \\ 0 & 0.5 \end{pmatrix}$$

$$q = (1 \cdot 1 + 2 \cdot 1, 1 \cdot 0 + 2 \cdot 1) = (3, 2), \quad k = (2, 1), \quad v = (2, 1)$$

One embedding has become three different vectors (Figure 3; here k and v happen to be equal, because W_K and W_V map this particular e to the same point).

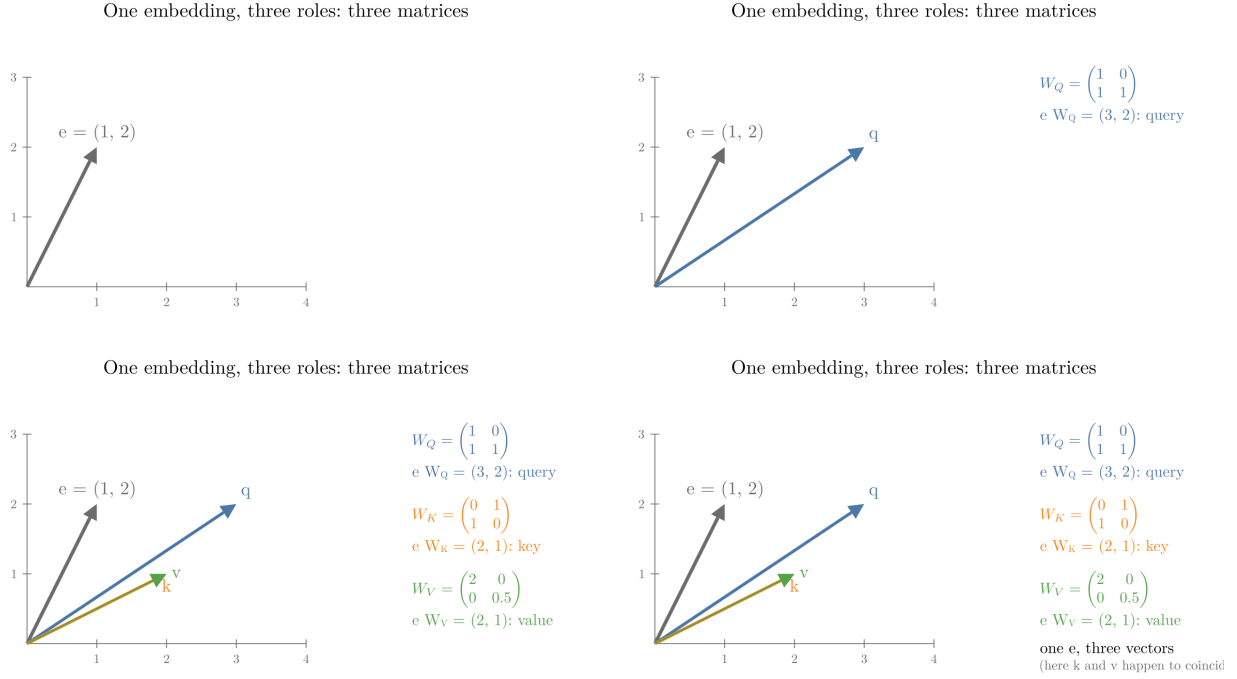


Figure 3: The embedding $e = (1, 2)$ multiplied by W_Q , W_K and W_V : each matrix moves the arrow to a new place

The same matrices for every word. The W_Q used for “money” is exactly the W_Q used for “bank” and for “grows”, number for number, and the same holds for W_K and W_V . Three words give $3 \times 3 = 9$ vectors from only three matrices.

Learned from data. Nobody chooses the numbers in W_Q , W_K and W_V . They start random. The model translates (or classifies) a sentence, makes mistakes, and backpropagation changes the matrices to reduce the loss; the next sentence gets slightly better query, key and value vectors. At the end of training the matrices hold the numbers that produce the best vectors for this task.

8.2 The refined computation

Key point: Same three steps as before, with queries in the dot product, keys as what they are compared with, and values in the weighted sum.

The computation of section 4 stays the same; each use of an embedding is replaced by the vector for its role. For “bank”:

$$s_{2j} = q_{\text{bank}} \cdot k_j, \quad w_{2j} = \text{softmax}_j(s_{2j}), \quad y_{\text{bank}} = \sum_j w_{2j} v_j$$

8.3 All words at once

Key point: $Q = XW_Q$, $K = XW_K$, $V = XW_V$, and then $Y = \text{softmax}(QK^\top) V$. Still fully parallel.

The matrix form of section 5 carries over (Figure 4, and SLP3 eq. 7.33):

1. **In words:** multiply the embedding matrix by the three weight matrices; compare every query with every key in one product; softmax each row; mix the values.
2. **Formula:**

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad Y = \text{softmax}(QK^\top) V$$

3. **Example:** for “money bank grows” with 50-number embeddings and 50×50 matrices: X is 3×50 ; Q , K and V are 3×50 ; $QK^\top$ is 3×3 , one score for every pair of words; Y is 3×50 . The Notebook checks that the matrix form equals the word-by-word loop.

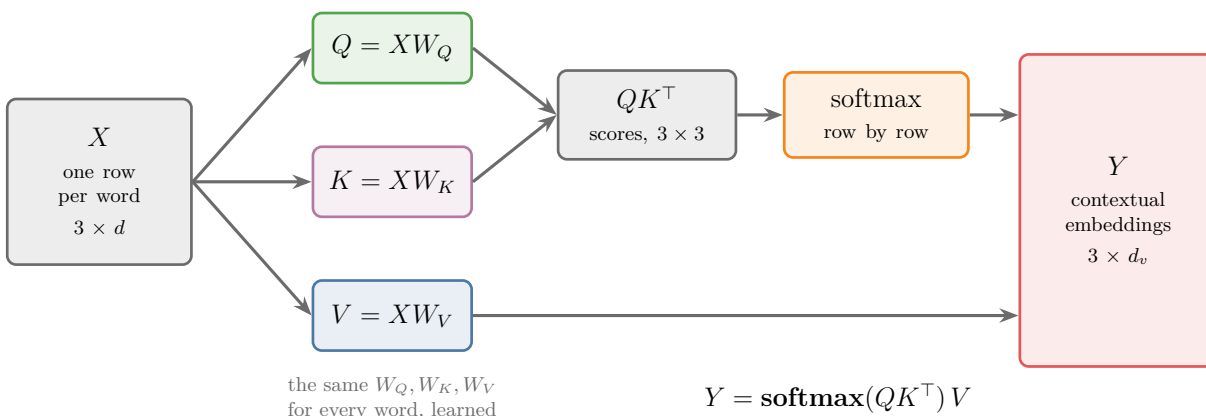


Figure 4: Self-attention with learned matrices, for a whole sentence at once

The order matters: $QK^\top$, queries times the transpose of the keys, so that entry (i, j) is $q_i \cdot k_j$.

The self-attention inside a transformer works this way, with one more step: the scores are divided by $\sqrt{d_k}$ before the softmax. Why that scaling is needed is the subject of the [scaled dot-product attention Note](#).

9. Learned matrices on real reviews

Key point: On IMDB movie reviews, with the same frozen embeddings, self-attention with learned W_Q , W_K , W_V reaches 0.84 test accuracy against 0.69 without parameters. The learned weights concentrate on the word that matters for the task: “terrible”.

9.1 The experiment

Key point: Two models identical except for the three matrices, which start at the identity, so both begin as the same function.

Section 6 claimed that learnable matrices make the contextual embeddings task-specific. The Notebook tests the claim on sentiment analysis of IMDB movie reviews (the [RNN sentiment analysis Note](#)): 25,000 training and 25,000 test reviews, each an **observation** (one record of the data), with the sentiment, positive or negative, as the **target** (the output we predict). The words are the 10,000 most frequent, and each review is cut to 200 words.

- **Static embeddings:** 50 numbers per word, learned from the training reviews themselves by the counting method of the [what is self-attention Note](#), section 4.2, then **frozen**, so they cannot adapt to the task. They do capture meaning: the nearest words to “great” are wonderful, fine, fantastic, excellent, marvelous, superb.
- **Model:** embeddings $\rightarrow$ self-attention $\rightarrow$ the average of the contextual embeddings over the words of the review $\rightarrow$ one sigmoid node.
- **Fixed:** self-attention without parameters, $Y = \text{softmax}(XX^\top)X$. The only trainable weights are the 51 of the final node.
- **Learned:** $Y = \text{softmax}(QK^\top)V$ with W_Q, W_K, W_V of size 50×50 , which adds 7,500 trainable weights. The three matrices start at the identity matrix, so at the start the learned model computes exactly what the fixed model computes.

Both train with Adam for 5 epochs, batch size 64, 3 runs each with different seeds.

9.2 Results

Key point: The learned model is about 15 points more accurate, in every run.

Model	Trainable weights	Test accuracy (mean of 3 runs)
Fixed: no parameters	51	0.69
Learned W_Q, W_K, W_V	7,551	0.84

The three runs of each model agree to within 0.004. Both models have exactly the same information: the same frozen embeddings and the same reviews. The only difference is that the learned model can change how each word looks for, and mixes in, the other words, based on the labelled reviews.

9.3 What the weights learned

Key point: Without parameters, each word mostly attends to itself. After training, almost every word attends to “terrible”.

What did the matrices learn? Figure 5 takes a real 10-word stretch from a test review, “and the rest of the cast rendered terrible performances the”, and shows the attention weights of both models.

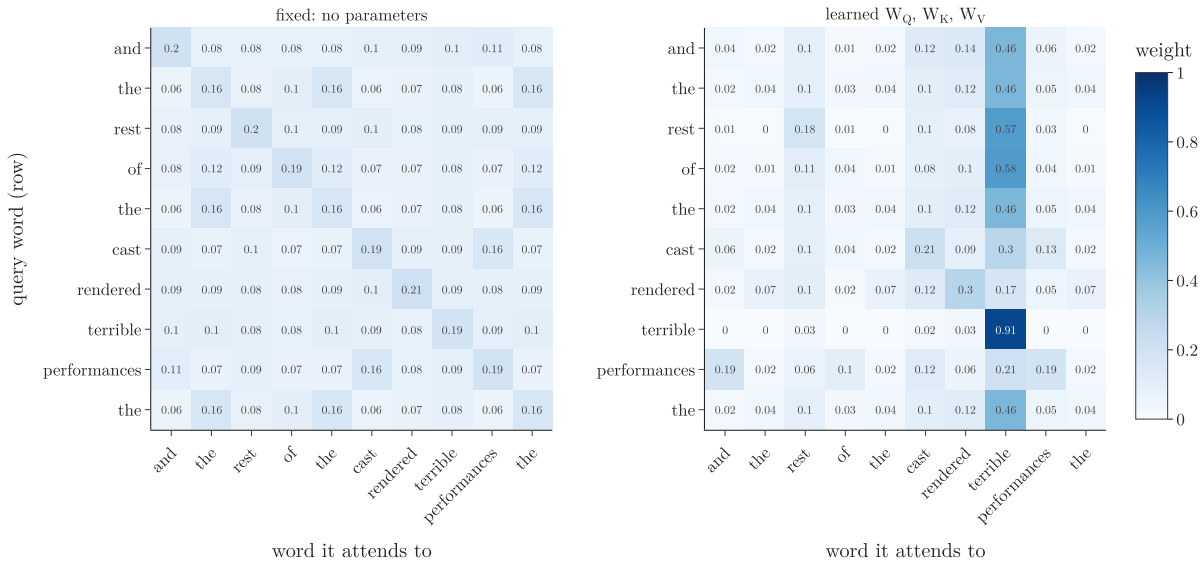


Figure 5: Attention weights on a real review. Left: without parameters, each word borrows mostly from itself and spreads the rest evenly. Right: with learned W_Q, W_K, W_V , almost every word borrows most from “terrible”, the word that decides the sentiment

- **Fixed (left):** the largest weight in each row is on the diagonal, about 0.2: each word is most similar to itself. The rest is spread evenly, about 0.08 per word. The weights know nothing about sentiment.
- **Learned (right):** the column of “terrible” lights up. “Rest” puts 0.57 of its weight on “terrible”, “of” 0.58, “the” 0.46, and “terrible” itself 0.91. After averaging, the review’s vector is dominated by the meaning of “terrible”, which is exactly what a sentiment classifier needs.

For this task, the learned matrices turned self-attention into a detector of sentiment words. A translation model trained on the same embeddings would learn different matrices, and attend differently. These are the task-specific contextual embeddings of section 6, measured.

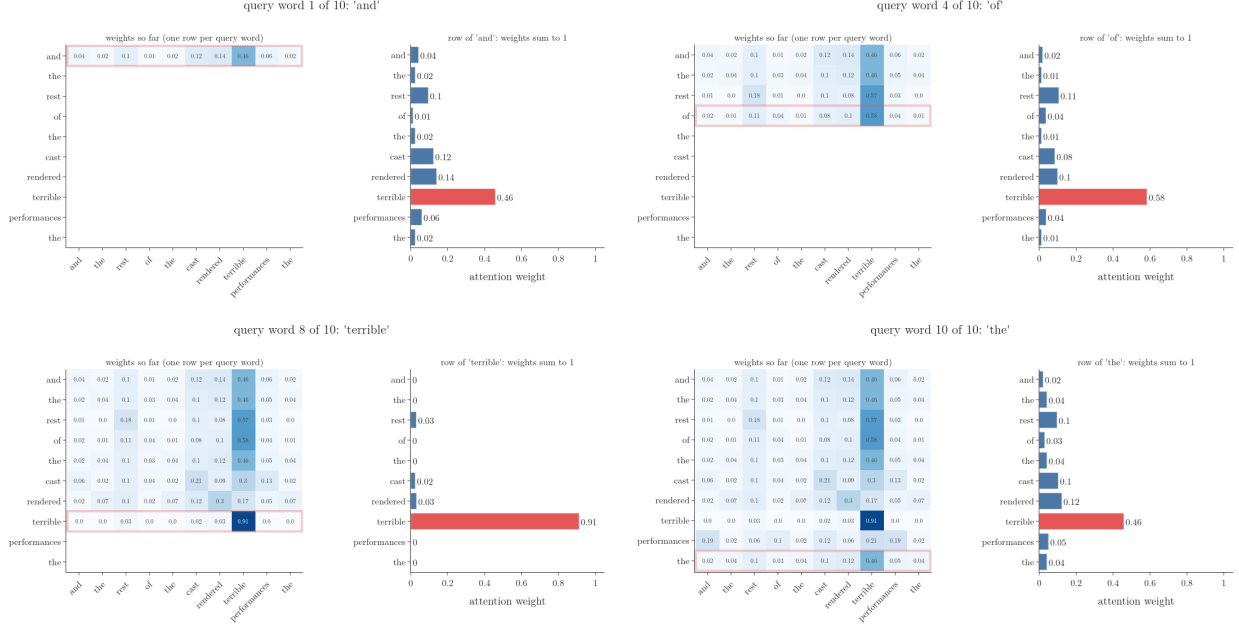


Figure 6: The learned weights forming one query word at a time. Left: the weight matrix filling row by row. Right: the current query word’s weights, which sum to 1; the largest is red

10. Summary

Step	Simple version	Refined version (used in transformers)
Vectors per word	the embedding e_i , used in all three roles	query $q_i = e_i W_Q$, key $k_i = e_i W_K$, value $v_i = e_i W_V$
Scores	$s_{ij} = e_i \cdot e_j$	$s_{ij} = q_i \cdot k_j$
Weights	softmax over j	softmax over j
Output	$y_i = \sum_j w_{ij} e_j$	$y_i = \sum_j w_{ij} v_j$
Whole sentence	$Y = \text{softmax}(X X^\top) X$	$Y = \text{softmax}(Q K^\top) V$
Parameters	none: general contextual embeddings	W_Q, W_K, W_V , learned: task-specific

- A word’s contextual embedding is a weighted sum of the vectors of all the words in its sentence.
- The weights are similarities (dot products) normalised by a softmax, so each row sums to 1.
- All words are processed at once with matrix products, which runs in parallel on a GPU but ignores word order.
- Learned matrices W_Q, W_K, W_V , the same for every word, make the result task-specific: on IMDB, 0.84 accuracy against 0.69, with attention concentrated on sentiment words.
- The transformer adds one more step, dividing the scores by $\sqrt{d_k}$ (the [scaled dot-product attention Note](#)).

11. Sources

Built from

- CampusX, “Self Attention in Transformers | Deep Learning | Simple Explanation with Code!”, YouTube, https://www.youtube.com/watch?v=-tCKPl_8Xb8
- Sanderson, G. (3Blue1Brown), “Attention in transformers, step-by-step | Deep Learning Chapter 6”, 2024, [3blue1brown.com/lessons/attention](https://www.youtube.com/watch?v=eMlx5fFN5:52-10:03), <https://www.youtube.com/watch?v=eMlx5fFN5:52-10:03> (an imagined head in which nouns’ queries look for preceding adjectives and adjectives’ keys answer; the attention pattern drawn as a grid of dots).

Other references

- Jurafsky, D. and Martin, J. H. *Speech and Language Processing*, 3rd ed. draft (19 August 2026), web.stanford.edu/~jurafsky/slp3. Chapter 7, section 7.1 (simplified attention: dot-product scores, softmax, weighted sum, eqs. 7.6–7.8; the three roles query, key and value and the matrices W_Q, W_K, W_V , eq. 7.9); section 7.3 (all positions at once: $Q = XW_Q, K = XW_K, V = XW_V$ and $QK^\top$, eq. 7.33 and Figure 7.9).
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. Section 3.2 (queries, keys and values; the output is a weighted sum of the values).
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y. and Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. *ACL 2011*. The IMDB review dataset.

- Tatoeba (tatoeba.org) English–French sentence pairs, prepared by manythings.org/anki and distributed for the Keras examples. Licence CC BY 2.0. Source of the embeddings of sections 3 to 5.

12. Key terms

Term	Meaning
Self-attention	A mechanism that gives each word of a sequence a new vector: a weighted sum over all the words, with weights from query–key similarity
Score	The dot product of a query with a key: how well two words match
Attention weight	A score after the softmax; the weights of one word sum to 1
Query	The vector of the word whose new vector is being computed; it is compared with every key
Key	The vector a word offers to be compared with a query
Value	The vector a word contributes to the weighted sum
W_Q, W_K, W_V	The learned matrices that turn an embedding into its query, key and value vectors; the same for every word
General contextual embedding	A contextual embedding computed without learned parameters, the same for every task
Task-specific contextual embedding	A contextual embedding computed with parameters learned from the task’s data
Observation	One record of the data: here, one review
Target	The output we predict: here, the sentiment of a review